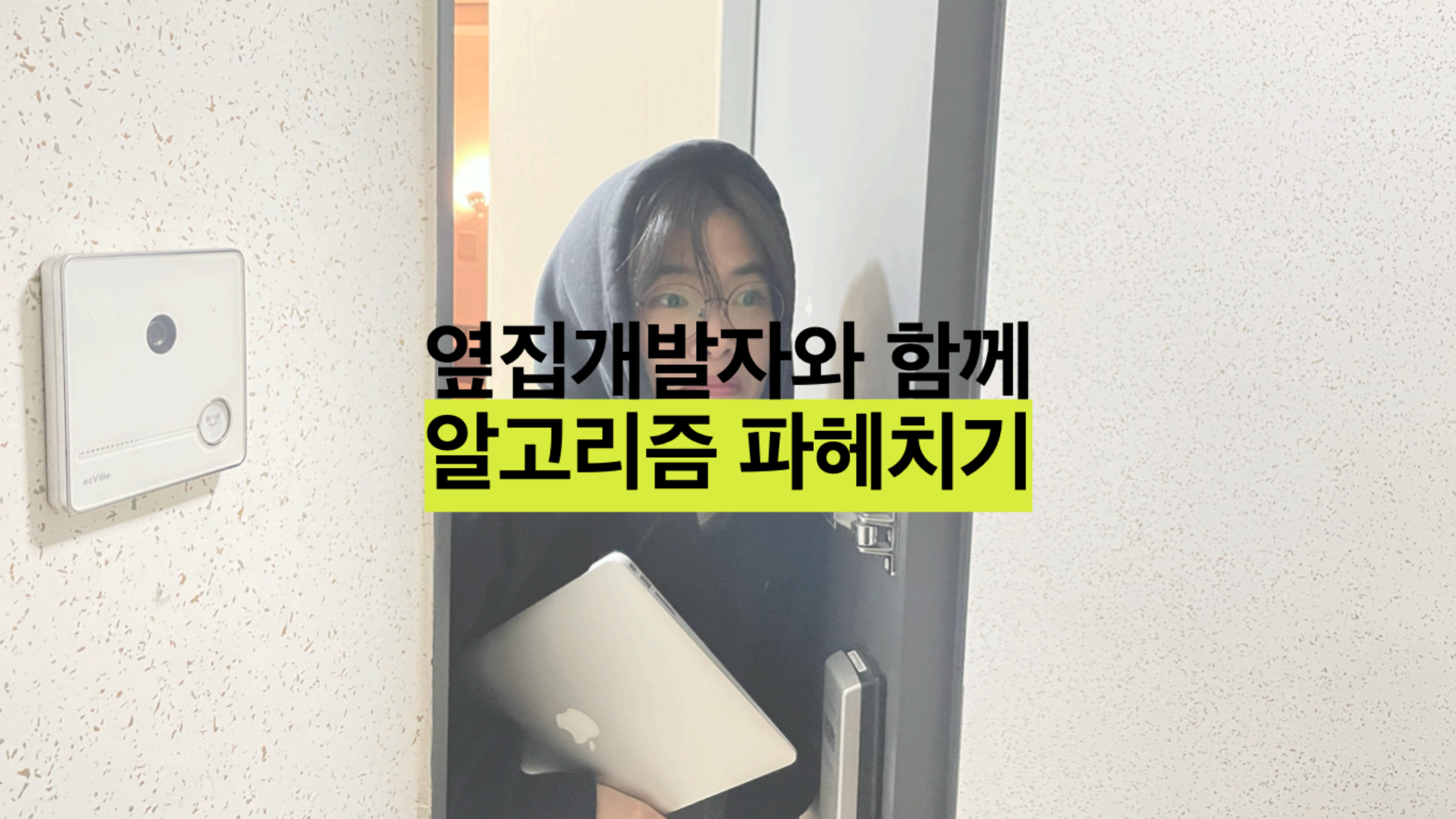

옆집개발자와 함께
알고리즘 파헤치기

국민건강보험공단 전산전문교육

미니 프로젝트 기반 알고리즘 실전 학습

청구 데이터 점검기를 만들며 배우는 3일
찾고 · 대조하고 · 집계하고 · 골라내는 알고리즘

2026.11.09 – 11.11 | 강사 김송아

0.

들어가며

데이터가 중요한 회사에서,
AI 시대에 알고리즘이 필요한 이유

오늘 우리 공단에 들어온 청구는 몇 건일까요?

① 1만 건 ② 10만 건 ③ 100만 건 ④ 그 이상

하루 평균 (2025년 심사 청구 기준)

431 만 건

1년이면 **약 15.7억 건**, 진료비는 **약 140조 원**

출처: 건강보험심사평가원 보건의료빅데이터개방시스템 · 총진료비 현황 (2025년 심사결정분 1,573,740천 건 ÷ 365일)

숫자로 보는 건강보험

5,140만명
건강보험 적용인구
2025년 · 51,395천 명

10.4만곳
청구 요양기관
2025년 · 104,285곳

15.7억건
연간 청구 건수
2025년 · 1,573,740천 건

140조원
연간 진료비
2025년 · 1,408,435억 원

사람 한 명이 청구를 **1초에 1건씩** 쉬지 않고 확인하면, 하루치(431만 건)만 봐도 **약 50일**이 걸립니다.

출처: 적용인구 — 국민건강보험공단 경영공시 · 연도별 적용인구 현황 / 요양기관·청구·진료비 — 건강보험심사평가원 보건의료빅데이터개방시스템
※ 심평원 수치는 심사결정분 기준이며, 건강보험 외 의료급여·보훈·자동차보험 청구를 포함합니다.

공단의 데이터 업무는 결국 네 가지 동사

이 네 동사가 3일 동안 배울 알고리즘의 지도입니다.

찾기

특정 사람,
특정 기간의 청구

Day 2

대조하기

자격이 있는 사람의
청구인지 확인

Day 1

집계하기

월별 · 기관별
진료비 합계

Day 2

골라내기

중복된 청구,
튀는 청구

Day 3

AI에게 시키면 몇 초 만에 코드가 나옵니다

```
// 질문
"100만 건의 청구 데이터에서 중복 청구를 찾는 Java 코드를 짜줘"

// 응답 (3초)
for (i = 0; i < claims.size(); i++) {
  for (j = i + 1; j < claims.size(); j++) {
    if (claims.get(i).equals(claims.get(j))) result.add(...);
  }
}
```

테스트용 100건에서는 **정답이 나옵니다.** 이 코드, 실제 데이터에서도 믿어도 될까요?

맞아 보이는 코드의 두 가지 함정

① 느린 코드

1만 건 테스트: **1초**

100만 건 실데이터: **약 9시간**

데이터가 100배가 되면 시간은 **1만 배**가 됩니다. (이중 반복문)

② 경계에서 틀리는 코드

입원 A: 3/1 ~ **3/10**

입원 B: **3/10** ~ 3/15

퇴원일 = 다음 입원일이면 **겹치는 걸까요?** AI는 자신 있게 답하지만 기준을 알려준 적이 없습니다.

※ 이중 반복문 기준. 1만 건 약 0.9초, 10만 건 약 5분을 직접 측정했고, 100만 건은 그 값의 100배로 추정한 시간입니다.

AI 시대에 필요한 건 세 가지 힘

직접 다 짜는 힘이 아니라, AI가 짠 코드를 **다룰 줄 아는 힘**입니다.

1 **문제를 쪼개는 힘**
이건 대조 문제다. 이건 기간 문제다.
어떤 알고리즘?

2 **효율을 판단하는 힘**
데이터가 10배 늘면 몇 배 느려지나.
얼마나 빠른가?

3 **검증하는 힘**
이 코드가 틀리는 반례는 무엇인가.
정말 맞는가?

이 과정은 이 세 가지를 **3일 동안 손으로 익히는 시간**입니다.

3일 동안 우리가 만들 것은
청구 데이터 점검기입니다

찾고 · 대조하고 · 집계하고 · 골라내는 프로그램

3일 동안 청구 데이터 점검기를 만듭니다

[점검 결과] 청구 100,000건 / 가입자 20,000명 완성 화면

- 중복 청구 37건
- 자격 없는 사람의 청구 12건
- 자격 기간 밖 진료 58건
- 중복 입원 기간 9건
- 진료비 상위 10건 ..... 목록 출력
- 동일인 의심 쌍 21쌍
- 이상 청구 44건
- 청구 급증일 3일

※ 교육 때 제공하는 가상 데이터로 점검기를 돌린 결과입니다. 3일 뒤 여러분의 점검기에서 같은 숫자가 나옵니다. 실제 데이터는 사용하지 않습니다.

가상 건강보험 데이터 3종

데이터	주요 항목
자격 이력	가입자ID 이름 생년월일 취득일 상실일
진료 청구	청구ID 가입자ID 요양기관 진료일 진료비
입원 청구	청구ID 가입자ID 요양기관 입원일 퇴원일 진료비

강사가 제공

- 뼈대 코드 (데이터 불러오기, 결과 출력)
- 테스트 데이터

여러분이 채움

- 블록마다 **함수 하나씩**
- 3일 동안 기능이 하나씩 붙습니다

날마다 점검기에 기능이 붙습니다

	주제	🔧 추가되는 기능	업무 동사
Day 1	해시	중복 청구, 자격 없는 사람의 청구	대조하기
Day 2	정렬·이분 탐색, 구간, 누적합	특정 기간 조회, 자격 기간 밖 진료, 중복 입원, 월별 집계	찾기 · 집계하기
Day 3	힙, 편집 거리, 이상치	상위 N건, 동일인 판별, 튀는 청구	골라내기

Day 1
효율의 감각
시간 복잡도 · 해시

▶

Day 2
기간과 집계
정렬 · 구간 · 누적합

▶

Day 3
추출·탐지와 완성
힙 · 편집 거리 · 이상치

기본 트랙 / 도전 트랙, 골라서 진행합니다

	설계	구현	검증
기본 트랙	본인	**AI**	본인 (반례, 속도)
도전 트랙	본인	본인	본인 + AI 리뷰

> 두 트랙 모두 **설계와 검증은 본인**의 몫입니다. 트랙은 Sprint마다 바꿀 수 있습니다.

AI는 채팅창에서만 씁니다

사용 규칙

- 외부 LLM(ChatGPT, Gemini, Claude 등) **복사·붙여넣기**만 사용
- IDE 플러그인, 에이전트는 **사용 안 함**
- 이유: 사내 **NHIS-MATE**와 같은 조건에서 연습해야 돌아가서 바로 쓸 수 있습니다

"함수 완성"의 기준

- ✔ 테스트 데이터를 통과한다
- ✔ **반례를 1개** 직접 만들어 본다
- ✔ 대용량 데이터에서 **시간 안에** 끝난다

함께 실습 환경을 맞춥니다

- [ ] **IntelliJ IDEA 설치**
  Community 버전을 내려받아 설치하고 실행
  STEP 1

- [ ] **JDK 17 설치**
  설치 후 터미널에서 `java -version` 으로 17인지 확인
  STEP 2

- [ ] **뼈대 코드와 가상 데이터 내려받기**
  안내하는 링크에서 zip 다운로드 후 프로젝트로 열기
  STEP 3

- [ ] **`Main` 실행해서 빈 점검 리포트 확인**
  화면에 리포트 틀이 나오면 준비 완료
  STEP 4

- [ ] **외부 LLM 접속 확인**
  각자 계정으로 로그인, 채팅창에서 질문이 되는지 확인
  STEP 5

다음 시간

1장. 프로젝트 킥오프

가상 데이터를 살펴보고,
AI가 짠 점검 코드에서 **느린 부분과 틀린 반례**를 직접 찾아봅니다.

Day 1

국민건강보험공단 전산전문교육 · 미니 프로젝트 기반 알고리즘 실전 학습

Day 1
효율의 감각

프로젝트 킥오프 · 시간 복잡도 · 해시 · Sprint 1

1.

프로젝트 킥오프

데이터를 열어 보고,
AI가 준 코드를 의심해 봅니다

점검기를 만들기 전에
데이터부터 열어 봅니다

파일 3종 · 모두 가상 데이터입니다

파일은 3종입니다

qualifications.csv · 자격 이력

```
memberId,name,birthDate,startDate,endDate
M000001,김가온,1985-03-12,2019-04-01,
M000002,이서연,1992-11-03,2021-01-01,2024-06-30
M000002,이서연,1992-11-03,2024-09-01,
```

outpatient.csv · 진료 청구

```
claimId,memberId,institution,visitDate,fee
C0000001,M000001,한빛의원,2026-03-04,12500
C0000002,M000001,한빛의원,2026-03-04,12500
C0000003,M000002,푸른약국,2026-02-20,4800
```

inpatient.csv · 입원 청구

```
claimId,memberId,institution,admitDate,dischargeDate,fee
H0000001,M000002,새봄병원,2026-02-10,2026-02-17,1850000
H0000002,M000002,새봄병원,2026-02-17,2026-02-20,640000
```

한 가입자가 여러 행을 가질 수 있고, endDate가 비어 있으면 자격이 유지 중입니다. 실제 데이터는 쓰지 않습니다.

눈으로 찾아보세요 — 숨은 문제

1. **가입자 M000002의 자격 이력은 몇 행인가요?**
   지금도 자격이 있나요?
2. **endDate가 비어 있는 행은 무슨 뜻일까요?**
3. **C0000001과 C0000002는 같은 청구일까요, 다른 청구일까요?**
4. **H0000001의 퇴원일과 H0000002의 입원일이 같습니다. 겹치는 걸까요?**

기간은 시작일과 종료일을 모두 포함합니다

이 과정의 날짜 규칙

- 날짜 형식은 `yyyy-MM-dd`
- 3/1 ~ 3/10 이면 **3/10도 포함**
- endDate가 비어 있으면 **자격 유지 중**

입원 A 3/1~3/10, 입원 B 3/10~3/15

3/10 하루가 겹칩니다. 이 기준을 정해 두면 **겹침 판정**이 흔들리지 않습니다.

이 규칙은 Day 2 구간 알고리즘에서 다시 나옵니다.

3일 동안 찾을 문제 지도

숨어 있는 문제	점검 기능	배우는 곳
같은 청구가 두 번 들어옴	중복 청구 찾기	3장 해시
자격 이력에 없는 가입자의 청구	자격 없는 사람의 청구 찾기	3장 해시
특정 기간의 청구가 필요함	특정 기간의 청구 찾기	5장 정렬·이분 탐색
자격 기간 밖에서 받은 진료	자격 기간 밖 진료 찾기	6장 구간
입원 기간이 겹침	중복 입원 기간 찾기	6장 구간
월별·기간별 합계가 필요함	월별·기간별 청구 집계	7장 누적합
진료비가 가장 큰 청구	진료비 상위 N건 추출	9장 힙
표기만 다른 같은 사람	동일인·중복 데이터 판별	10장 편집 거리
갑자기 튀는 청구	튀는 청구 찾기	11장 이상치

프로젝트는 이렇게 생겼습니다

```
claim-checker/
├ data/
│  ├ small/   csv 3종 (눈으로 확인)
│  └ large/   csv 3종 (속도 비교)
├ src/main/java/checker/
│  ├ Main.java
│  ├ model/   Qualification, Outpatient,
│  │          Inpatient (record)
│  ├ DataLoader.java
│  ├ ReportPrinter.java
│  └ checks/  점검 함수 (처음엔 비어 있음)
└ src/test/java/   기대 결과 테스트
```

Main의 흐름

- 1. DataLoader가 CSV 3종을 읽는다
- 2. 점검 함수를 차례로 호출한다
- 3. ReportPrinter가 리포트를 출력한다

함수 하나가 기능 하나입니다

```
public class DuplicateClaims {
    // TODO [3장] 중복 청구의 청구ID 목록을 반환
    public static List<String> find(List<Outpatient> claims) {
        return List.of();
    }
}
```

> 처음에는 모두 비어 있고, 날마다 하나씩 채웁니다. 채울 때마다 리포트의 숫자가 바뀝니다.

먼저 실행해 봅니다

- [ ] **Main 실행**
  모든 항목이 0건인 빈 리포트가 나오면 정상
  STEP 1

- [ ] **small/ 데이터가 읽혔는지 건수 확인**
  자격 이력, 진료 청구, 입원 청구 건수가 나오는지
  STEP 2

- [ ] **테스트 실행**
  전부 실패(빨간불)가 정상입니다. 기능을 채울 때마다 초록불로 바뀝니다
  STEP 3

AI에게 시켜 봅니다

외부 LLM 채팅창에 아래 프롬프트를 붙여넣으세요.

```
아래 데이터에서 중복 청구를 찾는 Java 메서드를 작성해줘.

record Outpatient(String claimId, String memberId, String institution,
                  LocalDate visitDate, long fee) {}

입력: List<Outpatient>
출력: 중복 청구의 claimId 목록
```

"중복"이 무엇인지는 **일부러 알려주지 않았습니다.**

받은 코드를 넣고 small 데이터로 돌립니다

```
static List<String> findDuplicateClaims(List<Outpatient> claims) {
    List<String> result = new ArrayList<>();
    for (int i = 0; i < claims.size(); i++) {
        for (int j = 0; j < claims.size(); j++) {
            Outpatient a = claims.get(i), b = claims.get(j);
            if (i != j
                    && a.memberId().equals(b.memberId())
                    && a.institution().equals(b.institution())
                    && a.visitDate().equals(b.visitDate())) {
                result.add(a.claimId());
            }
        }
    }
    return result;
}
```

작은 데이터에서는 **정답처럼 보입니다.** 이제 large 데이터로 돌려 보세요. (AI의 실제 응답은 매번 다릅니다. 위 코드는 예시입니다.)

청구 10만 건 · 이중 반복문

5분 11초

청구 건수	실행 시간	비교 횟수
1만 건	약 0.9초	약 1억 번
2만 건	약 3.6초	약 4억 번
4만 건	약 24.7초	약 16억 번
10만 건	**약 311초**	약 100억 번

강사 노트북(Apple M1, JDK 21) 측정값입니다. 여러분의 노트북에서는 시간이 다르게 나옵니다. 직접 돌려서 기록해 보세요.

이 코드가 틀리는 입력을 만들어 보세요

힌트 질문

- 같은 청구가 **3건**이면?
- 같은 날 같은 기관인데 **진료비가 다르면**?
- 똑같은 행이 **두 번** 들어오면?

하는 일

- 반례 CSV를 직접 만든다
- 코드를 돌려 결과를 확인한다
- 어디가 이상한지 적는다

실제로 이렇게 틀렸습니다

반례	입력	코드의 결과
① 3건이 같은 청구	C1, C2, C3	`[C1, C1, C2, C2, C3, C3]` 같은 ID가 여러 번
② 같은 날 같은 기관, 진료비 다름	12,500원 / 48,000원	`[C1, C2]` 정상 진료를 중복으로 (오탐)
③ 같은 행이 두 번	C1, C1	`[C1, C1]` 어느 쪽이 사본인지 모름
④ 청구 1건 / 빈 목록	-	`[]` 문제 없음

> 작은 정상 데이터만 돌렸다면 **못 찾았을 오류**입니다. 반례를 직접 만들어 봐야 보입니다.

"중복"의 기준은 사람이 정합니다

> **가입자ID · 요양기관 · 진료일 · 진료비**가 모두 같으면 중복 청구로 봅니다.
> 먼저 들어온 1건은 **원본**으로 두고, 나머지를 중복으로 출력합니다.

AI는 이 기준을 모르는 채로 코드를 썼습니다. **기준을 정하는 건 설계이고, 설계는 본인의 몫**입니다. (0.4 트랙)

느리고, 틀립니다. 그래서?

느리다

데이터가 10배가 되면 시간은 100배가 됩니다.

→ **2장 시간 복잡도**에서 이유를 확인하고, **3장 해시**로 고칩니다.

틀린다

기준을 알려주지 않았고, 반례를 돌려 보지 않았습니다.

→ **설계와 검증은 본인**입니다. 3일 내내 반복합니다.

2.

시간 복잡도

데이터가 늘면 코드는
얼마나 느려질까요

청구가 2배가 되면
점검 시간은 몇 배가 될까요?

① 2배

② 4배

③ 코드에 따라 다르다

Big-O는 증가 속도를 말합니다

복잡도	읽는 법	n = 1만	n = 10만	n = 100만
O(1)	크기와 무관	1	1	1
O(log n)	아주 천천히 늘어남	약 13	약 17	약 20
O(n)	비례	1만	10만	100만
O(n log n)	비례보다 약간 더	약 13만	약 166만	약 1,990만
O(n^2)	제곱	**1억**	**100억**	**1조**

> n이 10배가 되면 O(n)은 10배, O(n^2)은 **100배**가 됩니다.

코드를 보고 반복문을 세어 판단합니다

O(n) — 한 번 훑기

```
for (Outpatient c : claims) {
    total += c.fee();
}
```

O(n^2) — 두 겹

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        ...
    }
}
```

한 겹처럼 보여도 숨은 반복문이 있습니다

```
for (Outpatient c : claims) {                  // n번
    if (!qualList.contains(c.memberId())) {    // contains가 목록을 처음부터 훑음 → m번
        ...
    }
}
```

List.contains

목록을 처음부터 훑습니다.

전체 **O(n × m)**

HashSet.contains

칸을 바로 계산합니다.

전체 **O(n)**

AI가 준 코드에서 가장 자주 놓치는 부분입니다. 채팅창에 "이 코드의 시간 복잡도를 설명해 줘"라고 물어보세요.

실험: 대용량 대조

진료 청구 **100,000건** 중에서, 자격 이력(가입자 **20,000명**)에 없는 가입자의 청구를 찾으세요.

1. **예측하기**
   이중 반복문은 몇 초, 해시는 몇 초일까요? 먼저 적어 보세요
2. **실행하기**
   `Benchmark`를 실행해서 두 방식의 시간을 기록합니다
3. **비교하기**
   예측과 실제의 차이, 두 방식의 배율을 확인합니다

같은 문제, 약 1,000배 차이

이중 반복문

약 4.5~6.8초

청구 10만 × 자격 2만, 최대 약 20억 번 비교

해시 (HashSet)

약 6ms

약 12만 번 확인

강사 노트북(Apple M1, JDK 21) 측정값이며, 첫 실행이 가장 느립니다. 여러분의 결과와 비교해 보세요.

규모를 키우면 차이가 벌어집니다

청구 건수	이중 반복문 (1.3의 코드)	해시
1만 건	약 0.9초	약 0.8ms
2만 건	약 3.6초	-
4만 건	약 24.7초	-
10만 건	**약 311초 (5분 11초)**	**약 18ms**
100만 건	약 9시간 (10만 건 값의 100배로 추정)	**약 192ms**

이중 반복문은 n이 2배가 되면 이론상 4배, 실측에서는 4~7배가 나왔습니다. 해시는 100만 건에서도 1초가 걸리지 않습니다.

알고리즘을 바꾸면 시간이 바뀝니다

이중 반복문

5분 11초

▶

해시

18ms

컴퓨터를 바꾸거나 AI에게 더 좋은 코드를 조르는 게 아니라, **어떤 방법을 쓸지 고르는 것**이 사람의 일입니다. 다음 장에서 해시가 왜 이렇게 빠른지 봅니다.

3.

해시

찾는 데 걸리는 시간을
없앱니다

사물함처럼 바로 찾습니다

목록에서 찾기

1번부터 열어 보기

사물함 20,000개 → 평균 1만 번 확인

해시로 찾기

계산해서 바로 열기

이름 → 37번 칸 → 1번 확인

```
키 "M000123"  →  해시 함수  →  칸 번호 37  →  37번 칸만 확인
```

데이터가 2만 건이든 100만 건이든 찾는 횟수가 거의 같습니다(평균 O(1)). 대신 칸을 미리 만들어 두는 만큼 메모리를 더 씁니다. 서로 다른 키가 같은 칸이 나오는 것을 충돌이라고 하며, 같은 칸 안의 몇 개만 비교합니다.

Java에서는 HashSet과 HashMap

```
Set<String> members = new HashSet<>();
members.add("M000001");
members.contains("M000001");              // true — 바로 확인

Map<String, Integer> count = new HashMap<>();
count.merge("M000001", 1, Integer::sum);   // 가입자별 건수 세기
```

자료구조	용도
HashSet	"있는지 없는지" 확인, 중복 제거
HashMap	키로 값을 찾기, 키별로 세기·모으기

키가 되려면 equals와 hashCode가 같아야 합니다

```
record DupKey(String memberId, String institution, LocalDate visitDate, long fee) {}
```

record

내용이 같으면 같은 키로 보도록 **equals와 hashCode를 자동으로 만들어 줍니다.**

일반 클래스

두 메서드를 만들지 않으면 내용이 같아도 **다른 객체**로 취급되어 중복을 못 찾습니다.

AI가 준 코드에서 확인해 볼 점입니다.

패턴 ① 중복 검출

HashSet.add는 이미 있는 값이면 **false**를 돌려줍니다.

```
Set<DupKey> seen = new HashSet<>();
for (Outpatient c : claims) {
    boolean isNew = seen.add(키);   // 처음이면 true, 이미 있으면 false
}
```

> 한 번 훑으면서 "처음 본 것"과 "이미 본 것"을 가릅니다. **O(n)**

패턴 ② 두 데이터 대조

1. **기준 데이터를 HashSet에 담는다**
   한 번만 하면 됩니다 (m건)

2. **다른 데이터를 훑으면서 contains로 확인한다**
   n번, 각각 바로 확인

해시	이중 반복문
$O(m + n)$	$O(m \times n)$

🔧 중복 청구 찾기

> 기준: 가입자ID · 요양기관 · 진료일 · 진료비가 모두 같으면 중복. 먼저 들어온 1건은 원본.

```
public static List<String> find(List<Outpatient> claims) {
    Set<DupKey> seen = new HashSet<>();
    List<String> result = new ArrayList<>();
    for (Outpatient c : claims) {
        DupKey key = new DupKey(c.memberId(), c.institution(), c.visitDate(), c.fee());
        if (!seen.add(key)) {          // 이미 있으면 중복
            result.add(c.claimId());
        }
    }
    return result;
}
```

청구를 한 번만 훑으므로 O(n). 10만 건에서 약 18ms.

🔧 자격 없는 사람의 청구 찾기

> 자격 이력에 한 번도 나오지 않는 가입자의 청구. (기간이 맞는지는 6장에서 다룹니다.)

```
public static List<String> find(List<Outpatient> claims, List<Qualification> quals) {
    Set<String> members = new HashSet<>();
    for (Qualification q : quals) {
        members.add(q.memberId());
    }
    List<String> result = new ArrayList<>();
    for (Outpatient c : claims) {
        if (!members.contains(c.memberId())) {
            result.add(c.claimId());
        }
    }
    return result;
}
```

O(m + n). 자격 2만 + 청구 10만이면 약 12만 번 확인. 약 6ms.

1.3의 반례를 다시 돌려 봅니다

반례	입력	1.3의 코드	해시 풀이
① 3건이 같은 청구	C1, C2, C3	`[C1, C1, C2, C2, C3, C3]`	`[C2, C3]`
② 진료비가 다름	12,500 / 48,000	`[C1, C2]` 오탐	`[]`
③ 같은 행 두 번	C1, C1	`[C1, C1]`	`[C1]`
④ 1건 / 빈 목록	-	`[]`	`[]`

> 빨라진 것은 **해시를 써서**, 정확해진 것은 **기준을 정해서**입니다. 두 가지는 별개이므로 해시로 바꿔도 반례 검증은 다시 합니다.

4.

Sprint 1

해시로 점검기의
첫 두 기능을 완성합니다

정답을 받으면 남는 게 없습니다

"풀어줘"

정답 코드가 바로 나옵니다. 붙여넣으면 동작은 하지만 **왜 그런지는 내 것이 되지 않습니다.**

"힌트만 줘"

방향을 받고, **설계는 내가** 합니다. 사내 NHIS-MATE처럼 질문하고 답을 읽는 환경에서도 그대로 쓸 수 있는 습관입니다.

힌트를 받는 질문 4가지

① 방향만 물어보기

```
정답 코드는 주지 말고, 이 문제에
어떤 자료구조를 쓰면 좋을지
힌트만 줘.
```

② 단계별 힌트

```
힌트를 3단계로 줘. 방향 → 자료구조
→ 의사코드. 내가 "다음"이라고 할
때만 다음 단계를 줘. 코드는 쓰지 마.
```

③ 내 설계 확인받기

```
내 설계는 이거야: [설계 메모]
맞는 방향인지만 말해 줘. 고친
코드는 주지 마.
```

④ 막혔을 때

```
이 코드가 이런 결과를 내.
[기대 / 실제] 원인 후보를
3가지만 알려줘. 수정 코드는 주지 마.
```

이번 Sprint에서 만들 것

함수	설명	장
`DuplicateClaims.find`	중복 청구의 청구ID 목록	3.3
`NoQualificationClaims.find`	자격 이력에 없는 가입자의 청구ID 목록	3.4

> 외부 LLM은 **채팅창 복사·붙여넣기만** 사용합니다.

트랙을 고르세요

	설계	구현	검증
기본 트랙	본인	**AI**	본인 (반례, 속도)
도전 트랙	본인	본인	본인 + AI 리뷰

> 두 트랙 모두 **설계와 검증은 본인**의 몫입니다. 트랙은 Sprint마다 바꿀 수 있습니다.

코드부터 쓰지 말고 설계 메모부터

```
문제:          (한 줄로)
키(기준):     (무엇이 같으면 같은 것으로 볼까?)
자료구조:     (HashSet? HashMap? 왜?)
예상 복잡도: (O(?))
```

> 두 트랙 모두 먼저 적습니다. 질문 ③으로 방향을 확인받아도 됩니다.

함수 완성의 기준

- [ ] small 데이터 테스트를 통과한다
- [ ] 반례를 **1개 이상** 직접 만들어 본다
- [ ] large 데이터에서 **수 초 안에** 끝난다

먼저 끝났다면 도전 과제

1. **3건 이상 중복을 묶음으로 출력**
   `Map<DupKey, List<String>>`로 그룹화
2. **재전송과 내용 중복을 구분**
   같은 청구ID가 두 번 들어온 경우를 따로 출력
3. **요양기관을 뺀 기준을 시험**
   가입자·진료일·진료비만 같아도 중복으로 보면 오탐이 얼마나 늘어나는지 비교
4. **가입자별 청구 건수 상위 5명**
   `HashMap`으로 세기 (9장 힙의 준비)

Day 1 정리

오늘 점검기에 두 기능이 붙었습니다

중복 청구 · 자격 없는 사람의 청구

내일은 **기간**을 다룹니다. "시작일·종료일 모두 포함" 규칙이 다시 나옵니다.

국민건강보험공단 전산전문교육 · 미니 프로젝트 기반 알고리즘 실전 학습

Day 2
기간과 집계

정렬·이분 탐색 · 구간 알고리즘 · 누적합 · Sprint 2

5.

정렬 & 이분 탐색

정렬해 두면
찾기가 순식간입니다

같은 기간 조회를 천 번 하면?

선형 탐색

약 2.1초

매번 10만 건을 훑음

정렬 + 이분 탐색

약 0.4ms

정렬 1회(약 36ms)는 별도

청구 100,000건 · Apple M1, JDK 21 측정값. 교육장 노트북에서는 값이 다릅니다.

사전처럼 가운데부터 펼칩니다

사전에서 "하늘"을 찾을 때 첫 쪽부터 넘기지 않습니다. 가운데를 펼치고, 앞쪽인지 뒤쪽인지 알면 **나머지 절반을 통째로 버립니다.**

```
10만 건 → 5만 → 2.5만 → 1.25만 → ... → 1건      (절반씩 버리면 최대 약 17번)
```

데이터	최대 비교 횟수
10만 건	약 17번
100만 건	약 20번

> 전제: 목록이 **정렬되어 있어야** 합니다. 정렬되지 않았다면 "한쪽을 버리는" 판단이 성립하지 않습니다.

정렬은 한 번, 조회는 여러 번

	비용
정렬 (100,000건)	약 36ms, O(n log n)
선형 탐색 한 번	약 2.1ms, O(n)
이분 탐색 한 번	약 0.0004ms, O(log n)

> 약 **17번 이상** 조회하면 정렬한 쪽이 이득입니다. 조회가 한두 번이면 그냥 훑는 편이 낫습니다.

정렬한 뒤 데이터가 추가되면 정렬 상태가 깨집니다. 다시 정렬하거나 넣을 위치를 찾아서 넣어야 합니다.

Java에서 정렬하기

```
List<Outpatient> sorted = new ArrayList<>(claims);     // 원본은 그대로 두고 복사본을 정렬
sorted.sort(Comparator.comparing(Outpatient::visitDate));
```

> 객체 정렬(`List.sort`)은 **안정 정렬**입니다. 진료일이 같은 청구는 원래 순서가 유지됩니다.

찾는 건 값이 아니라 경계입니다

lowerBound(x)

진료일 **≥ x**인 **첫 위치**

→ 시작일의 위치

upperBound(x)

진료일 **> x**인 **첫 위치**

→ 종료일 **다음** 위치

기간 안의 청구 = `subList(lowerBound(시작일), upperBound(종료일))`
종료일을 **포함**하기 때문에 종료일에는 >를 쓰는 upperBound를 씁니다.

lowerBound 코드

```
static int lowerBound(List<Outpatient> a, LocalDate x) {
    int lo = 0, hi = a.size();          // [lo, hi) 반열린 구간: hi는 포함하지 않음
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;      // (lo + hi) / 2 는 값이 매우 크면 넘칠 수 있음
        if (a.get(mid).visitDate().isBefore(x)) lo = mid + 1;   // mid는 x보다 앞 → 오른쪽에 있음
        else hi = mid;                                           // mid가 후보 → 왼쪽까지 포함해서 계속
    }
    return lo;
}
```

upperBound는 조건만 `!a.get(mid).visitDate().isAfter(x)`로 바뀝니다.

이분 탐색의 흔한 실수

실수	결과	예방
`lo <= hi` 와 `hi = mid` 를 섞음	무한 반복	구간 표기 하나로 통일 `[lo, hi)`
`(lo + hi) / 2`	아주 큰 값에서 넘침	`(lo + hi) >>> 1`
lowerBound / upperBound 혼동	종료일 하루가 빠지거나 더 들어옴	종료일은 upperBound
정렬되지 않은 목록에 사용	엉뚱한 결과 (에러도 안 남)	정렬 여부 먼저 확인
`Collections.binarySearch` **그대로 사용**	같은 날짜가 여러 건이면 **어느 위치를 돌려줄지 보장하지 않음**	"첫 위치"가 필요하면 lowerBound 직접 구현

🔧 특정 기간의 청구 찾기

> 규칙: 시작일·종료일을 **모두 포함**. 시작일이 종료일보다 늦으면 빈 목록. 입력은 진료일 순으로 정렬되어 있다고 가정.

```
public static List<Outpatient> findByPeriod(
        List<Outpatient> sorted, LocalDate from, LocalDate to) {
    if (from.isAfter(to)) return List.of();
    return sorted.subList(lowerBound(sorted, from), upperBound(sorted, to));
}
```

조회 한 번에 O(log n). `subList`는 복사 없이 구간만 가리킵니다. 정렬은 호출하는 쪽에서 한 번만 합니다.

이렇게 검증했습니다

검증	결과
무작위 구간 2,000회, 선형 탐색 결과와 건수 비교	모두 일치
시작일 = 종료일 (하루)	해당 날짜 청구가 모두 나옴
시작일 > 종료일	빈 목록
데이터 범위 밖의 기간	빈 목록

Java 21에서 `--release 17`로 컴파일해서 실행했습니다.

6.

구간 알고리즘

기간이 겹치는지,
정확히 판단합니다

퇴원일이 3/10, 다음 입원일도 3/10.
겹치는 걸까요?

입원 A: 3/1 ~ 3/10 · 입원 B: 3/10 ~ 3/15

겹치지 않는 경우는 둘뿐입니다

A가 B보다 완전히 앞

A의 종료일 < B의 시작일

A가 B보다 완전히 뒤

B의 종료일 < A의 시작일

이 둘이 아니면 겹칩니다. 부분 겹침, 포함, 같은 기간이 **한 줄**로 처리됩니다.

```
static boolean overlaps(LocalDate s1, LocalDate e1, LocalDate s2, LocalDate e2) {
    return !s1.isAfter(e2) && !s2.isAfter(e1);        // s1 <= e2 그리고 s2 <= e1
}
```

실행해서 확인한 판정표

A	B	겹침	이유
3/1 ~ 3/10	3/10 ~ 3/15	**예**	3/10 하루가 겹침 (경계 포함)
3/1 ~ 3/10	3/11 ~ 3/15	아니오	하루 차이로 안 겹침 (이어지지만 겹치지는 않음)
3/1 ~ 3/31	3/5 ~ 3/6	**예**	포함 관계

흔한 실수	놓치는 경우
시작일끼리, 종료일끼리 같은지만 비교	일부만 겹치는 경우
`<` 와 `<=` 를 섞어 씀	경계 하루 (3/10)
한쪽 기준으로만 "안에 들어 있나" 확인	포함 관계, 양쪽에 걸친 경우

구간 병합: 정렬하고 한 번 훑습니다

입력	결과
1/1~1/10, 1/5~1/20, 1/21~1/25, 2/1~2/3	1/1~1/20, 1/21~1/25, 2/1~2/3 (3개)

> 1/20과 1/21은 하루 이어지지만 **겹치지는 않으므로** 합치지 않았습니다. 이어진 기간까지 합치려면 `next.start ≤ cur.end + 1`일로 바꿉니다. 어느 쪽이 맞는지는 **업무 규칙**이 정합니다.

순서: ① 시작일 순 정렬 → ② 첫 기간을 **지금 기간**으로 → ③ 다음 기간이 겹치면 **합치고**, 안 겹치면 지금 기간을 확정

병합 코드

```
static List<Period> merge(List<Period> periods) {
    if (periods.isEmpty()) return List.of();
    List<Period> sorted = new ArrayList<>(periods);
    sorted.sort(Comparator.comparing(Period::start));
    List<Period> result = new ArrayList<>();
    Period cur = sorted.get(0);
    for (int i = 1; i < sorted.size(); i++) {
        Period next = sorted.get(i);
        if (!next.start().isAfter(cur.end())) {                // 겹침 → 합침
            cur = new Period(cur.start(), max(cur.end(), next.end()));
        } else {                                               // 안 겹침 → 지금까지를 확정
            result.add(cur);
            cur = next;
        }
    }
    result.add(cur);
    return result;
}
```

시간 복잡도: 정렬 O(n log n) + 훑기 O(n)

스위핑: 훑으면서 겹친 개수를 셉니다

"동시에 입원 중인 환자가 가장 많은 날은?" 시작일에 **+1**, 종료일 **다음 날**에 **−1**을 놓고 날짜순으로 더합니다.

기간	시작 +1	끝 다음 날 −1
A: 3/1~3/10	3/1	3/11
B: 3/5~3/8	3/5	3/9
C: 3/9~3/12	3/9	3/13

```
3/1 (+1 → 1)   3/5 (+1 → 2)   3/9 (−1, +1 → 2)   3/11 (−1 → 1)   3/13 (−1 → 0)
```

> 가장 많을 때는 **2명**. 종료일을 **포함**하기 때문에 −1은 종료일 **다음 날**에 놓습니다. 개념을 익히는 용도이며 점검기 기능은 아닙니다 (도전 과제).

🔧 자격 기간 밖 진료 찾기

규칙

- 자격 이력이 **있는** 가입자가 자격 기간 **밖**에서 받은 진료
- 자격 이력이 아예 없으면 3.4의 몫 (이중으로 안 잡히게)
- 상실일이 비어 있으면 유지 중

접근

- ① 가입자별로 자격 기간을 모음
- ② 가입자별 기간을 **병합** (6.2)
- ③ 청구마다 진료일이 속하는지 **이분 탐색** (5장)

병합하는 이유: 기간이 서로 겹쳐 있으면 "시작일이 진료일 이하인 마지막 기간 하나만 보면 된다"는 이분 탐색의 전제가 깨지기 때문입니다.

자격 기간 밖 진료 코드 ① 가입자별로 모아 병합

```
public static List<String> outOfQualificationPeriod(
        List<Outpatient> claims, List<Qualification> quals) {
    Map<String, List<Period>> byMember = new HashMap<>();
    for (Qualification q : quals) {
        LocalDate end = q.endDate() == null ? LocalDate.MAX : q.endDate();   // 비어 있으면 유지 중
        byMember.computeIfAbsent(q.memberId(), k -> new ArrayList<>())
                .add(new Period(q.startDate(), end));
    }
    byMember.replaceAll((k, v) -> merge(v));                                 // 가입자별 기간 병합

    List<String> result = new ArrayList<>();
    for (Outpatient c : claims) {
        List<Period> periods = byMember.get(c.memberId());
        if (periods == null) continue;                                       // 자격 이력이 없으면 3.4의 몫
        if (!contains(periods, c.visitDate())) result.add(c.claimId());
    }
    return result;
}
```

청구 10만 건에서 약 14ms, **58건**을 찾았습니다.

자격 기간 밖 진료 코드 ② 속하는지 이분 탐색

```
static boolean contains(List<Period> merged, LocalDate d) {
    int lo = 0, hi = merged.size();          // 시작일이 d보다 늦은 첫 위치를 찾는다
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (!merged.get(mid).start().isAfter(d)) lo = mid + 1;
        else hi = mid;
    }
    int idx = lo - 1;                        // 시작일 ≤ d 인 마지막 기간
    return idx >= 0 && !d.isAfter(merged.get(idx).end());
}
```

> 5장의 이분 탐색과 같은 모양입니다. 병합되어 서로 겹치지 않기 때문에 "시작일이 d 이하인 마지막 기간" **하나만** 확인하면 됩니다.

🔧 중복 입원 기간 — 인접한 쌍만 보면 놓칩니다

```
A: 3/1  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~ 3/20
B:      3/5 ~~~ 3/8
C:                    3/15 ~~~ 3/18        ← B와는 안 겹치지만 A와는 겹침
```

시작일 순으로 정렬한 뒤 바로 앞 입원하고만 비교하면 C를 놓칩니다.

> 그래서 **지금까지 가장 늦게 끝나는 입원** 하나를 기억해 두고 비교합니다. 퇴원일 = 다음 입원일이면 겹치는 것으로 봅니다 (포함 규칙).

중복 입원 기간 코드

```
public static List<OverlapPair> overlappingAdmissions(List<Inpatient> admissions) {
    Map<String, List<Inpatient>> byMember = new LinkedHashMap<>();
    for (Inpatient a : admissions) byMember.computeIfAbsent(a.memberId(), k -> new ArrayList<>()).add(a);

    List<OverlapPair> result = new ArrayList<>();
    for (List<Inpatient> list : byMember.values()) {
        list.sort(Comparator.comparing(Inpatient::admitDate).thenComparing(Inpatient::dischargeDate));
        Inpatient latest = null;                          // 지금까지 가장 늦게 끝나는 입원
        for (Inpatient cur : list) {
            if (latest != null && !cur.admitDate().isAfter(latest.dischargeDate())) {
                result.add(new OverlapPair(latest.claimId(), cur.claimId()));
            }
            if (latest == null || cur.dischargeDate().isAfter(latest.dischargeDate())) latest = cur;
        }
    }
    return result;
}
```

입원 약 6,000건에서 약 2ms, **9건**을 찾았습니다. 이 중 3건은 퇴원일 = 다음 입원일인 경계 사례입니다. 3건 이상이 얽힌 경우 모든 쌍을 나열하지는 않습니다.

이렇게 검증했습니다

검증	결과
겹침 판정 3가지 (경계, 하루 차이, 포함)	판정표와 일치
구간 병합 예	3개로 병합
자격 기간 밖: 무작위 300회, 직접 비교와 비교	모두 일치
중복 입원: 무작위 300회, 모든 쌍을 비교한 결과와 비교	모두 일치
가상 데이터에 심어 둔 오류 (자격 기간 밖 58건, 중복 입원 9건)	정확히 찾음

Java 21에서 `--release 17`로 컴파일해서 실행했습니다.

7.

투 포인터 & 누적합

매번 다시 더하지 않고
한 번에 구합니다

기간 합계를 천 번 물어보면?

매번 14일치를 더할까요, 미리 더해 둘까요?

미리 더해 두면 빼기 한 번

```
일(번호)       :  0  1  2  3  4
일별 합계      : [3, 1, 4, 1, 5]
누적합 prefix : [0, 3, 4, 8, 9, 14]      ← 앞에 0을 하나 붙임
```

> 1일부터 3일까지의 합 = `prefix[4] − prefix[1]` = 9 − 3 = **6** (1 + 4 + 1)

일반식: **l일부터 r일까지의 합 = prefix[r + 1] − prefix[l]**

상황	방법
합계를 한 번만	그냥 더하기
기간을 바꿔 가며 여러 번	누적합
데이터가 계속 바뀜	누적합은 맞지 않음 (바뀔 때마다 다시 만듦)

누적합 코드

```
static long[] prefixSum(long[] a) {
    long[] p = new long[a.length + 1];                    // p[0] = 0
    for (int i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
    return p;
}

static long periodSum(long[] prefix, LocalDate base, LocalDate from, LocalDate to) {
    int l = (int) ChronoUnit.DAYS.between(base, from);
    int r = (int) ChronoUnit.DAYS.between(base, to);
    l = Math.max(l, 0);
    r = Math.min(r, prefix.length - 2);
    if (l > r) return 0;
    return prefix[r + 1] - prefix[l];                     // 빼기 한 번
}
```

만들 때 O(n), 이후 구간 합은 **O(1)**. 측정(청구 10만 건, 273일): 누적합 만들기 약 3.7ms, 기간 합 1,000회 약 **0.11ms**

두 목록을 한 번에 훑습니다

날짜순 청구 10만 건을 분기별 3개 기간으로 합산합니다. 기간마다 청구 전체를 훑으면 O(n × 기간 수)입니다.

원리

- 청구 쪽 포인터 `i` 는 **앞으로만** 움직임
- ① 시작일 전 청구는 건너뜀
- ② 종료일까지의 청구를 더함
- 다음 기간은 이어서 시작

전제

- 청구는 **날짜순**
- 기간은 날짜순이면서 **서로 겹치지 않음**
- 겹치면 이 방식은 맞지 않음

시간 복잡도: **O(n + 기간 수)**

투 포인터 코드

```
static long[] sumByPeriods(List<Outpatient> sortedClaims, List<Period> periods) {
    long[] result = new long[periods.size()];
    int i = 0;
    for (int k = 0; k < periods.size(); k++) {
        Period p = periods.get(k);
        while (i < sortedClaims.size() && sortedClaims.get(i).visitDate().isBefore(p.start())) i++;   // 시작 전 건너뛰기
        long sum = 0;
        while (i < sortedClaims.size() && !sortedClaims.get(i).visitDate().isAfter(p.end())) {          // 종료일까지 더하기
            sum += sortedClaims.get(i).fee();
            i++;
        }
        result[k] = sum;
    }
    return result;
}
```

투 포인터의 두 모양: ① 같은 방향으로 이동(이번 장, 11장 창) ② 양 끝에서 가운데로 이동(정렬된 목록에서 합이 목표값인 쌍). 이 과정은 ①만 다룹니다.

🔧 월별 집계 코드

```
public static Map<YearMonth, long[]> monthlySummary(List<Outpatient> claims) {
    Map<YearMonth, long[]> monthly = new TreeMap<>();            // 월 순서대로 정렬되어 나옴
    for (Outpatient c : claims) {
        long[] a = monthly.computeIfAbsent(YearMonth.from(c.visitDate()), k -> new long[2]);
        a[0]++;                                                  // 건수
        a[1] += c.fee();                                         // 진료비 합계
    }
    return monthly;
}
```

청구를 한 번만 훑으므로 O(n). 10만 건에서 약 8ms.

가상 데이터의 월별 결과

월	건수	진료비 합계(원)	월	건수	진료비 합계(원)
2026-01	10,631	1,641,642,800	2026-06	10,591	1,638,225,600
2026-02	11,270	1,727,897,600	2026-07	**12,326**	1,892,348,500
2026-03	10,808	1,669,509,700	2026-08	10,979	1,690,225,500
2026-04	10,433	1,605,944,700	2026-09	10,594	1,647,313,900
2026-05	**12,368**	1,906,138,100			

5월과 7월이 높은 것은 가상 데이터에 심어 둔 **청구 급증일** 때문입니다. 11장에서 찾습니다. 월별 합계를 모두 더하면 전체 합계와 같아야 합니다.

임의 기간은 누적합, 분기는 투 포인터

```
long[] daily  = dailyTotals(claims, BASE, 273);      // 일별 합계
long[] prefix = prefixSum(daily);                    // 한 번만 만들기
long sum = periodSum(prefix, BASE, from, to);        // 여러 번 물어도 O(1)
```

분기 (투 포인터)	진료비 합계(원)
1분기 (1/1~3/31)	5,039,050,100
2분기 (4/1~6/30)	5,150,308,400
3분기 (7/1~9/30)	5,229,887,900

어떤 방법을 고를까

요구	방법	시간
월·연도처럼 정해진 단위로 묶어 합계·건수	맵으로 그룹 집계 (`monthlySummary`)	O(n)
임의 기간의 합계를 여러 번	누적합	만들기 O(n), 조회 O(1)
날짜순 청구를 정해진 기간 목록으로 나눠 합산	투 포인터	O(n + 기간 수)

> "무엇을 묶는가, 몇 번 묻는가, 정렬되어 있는가"를 먼저 묻습니다.

이렇게 검증했습니다

검증	결과
월별 건수의 합 = 전체 건수, 월별 진료비의 합 = 전체 합계	일치
누적합 기간 합: 무작위 2,000회, 직접 더한 값과 비교	모두 일치
투 포인터 분기 합: 직접 더한 값과 비교	3개 모두 일치
투 포인터: 데이터가 없는 기간	0

Java 21에서 `--release 17`로 컴파일해서 실행했습니다.

8.

Sprint 2

Day 2의 알고리즘으로
점검기 네 기능을 완성합니다

이번에는 AI가 검사관입니다

	4장 (AI 활용 ①)	8장 (AI 활용 ②)
AI의 역할	힌트를 주는 코치	내 코드를 검사하는 **검사관**
내가 하는 일	설계, 구현	설계, 구현, **AI의 지적을 판단**
핵심	정답을 받지 않기	반례를 **많이, 빨리** 얻기

리뷰를 맡기는 질문 4가지

① 규칙 주고 코드 리뷰

```
[규칙]
아래 코드가 이 규칙대로 동작하는지
검토해 줘. 다르게 동작하는 부분만
줄 번호와 함께. 고친 코드는 주지 마.
```

② 반례 만들기

```
이 함수가 틀릴 만한 입력을 7개.
경계값 위주로. 입력마다 "기대하는
결과"와 "고른 이유"를 같이 적어 줘.
```

③ 시간 복잡도 점검

```
이 코드의 시간 복잡도를 설명해 줘.
숨은 반복문을 짚어 주고, 10만 건
일 때 문제가 없을지도 알려 줘.
```

④ 테스트 케이스

```
이 규칙으로 JUnit 테스트를 만들어 줘.
정상 1개, 경계 3개, 예외 1개.
```

규칙을 먼저 적어 줍니다

1.3에서 AI가 틀린 이유는 "중복의 기준"을 몰랐기 때문입니다. 기준 없이 검토를 시키면 AI는 자기 방식대로 추측해서 지적합니다.

이 과정의 규칙

- 기간은 시작일·종료일을 모두 포함한다
- 상실일이 비어 있으면 자격이 유지 중이다
- 중복 청구는 가입자·요양기관·진료일·진료비가 같은 것이다

규칙을 주면

- 지적이 규칙에 맞는지 **내가 판단**할 수 있습니다
- AI의 추측과 내 의도가 어긋나는 일이 줄어듭니다

반례를 만들 때 확인할 경계값

- [ ] 시작일 = 종료일 (하루짜리 기간)
- [ ] 퇴원일 = 다음 입원일 (하루 겹침)
- [ ] 하루 차이로 **안** 겹치는 경우
- [ ] 한 기간이 다른 기간을 통째로 포함
- [ ] 상실일이 비어 있는 경우
- [ ] 빈 입력, 데이터 1건 · 같은 날짜의 청구가 여러 건
- [ ] 연말·연초, 윤년(2/29)이 낀 기간

AI가 만든 반례도 검증 대상입니다

AI는 반례를 잘 만들지만 **"기대하는 결과"를 틀리게 적는 경우**가 있습니다. 코드가 맞고 기대값이 틀린 것일 수 있습니다.

1. **규칙에 비추어 기대값이 맞는지 직접 확인**
2. **그 반례로 코드를 돌려 본다**
3. **결과가 다르면 어느 쪽이 틀렸는지 규칙으로 판단**

사내 NHIS-MATE처럼 코드를 실행해 주지 않는 환경에서도 같습니다. AI는 입력을 만들어 주고, **실행과 판단은 내가** 합니다.

이번 Sprint에서 만들 것

함수	설명	장
`findByPeriod`	특정 기간의 청구	5.3
`outOfQualificationPeriod`	자격 기간 밖 진료의 청구ID	6.3
`overlappingAdmissions`	겹치는 입원의 쌍	6.4
`monthlySummary`	월별 건수와 진료비 합계	7.3

> 외부 LLM은 **채팅창 복사·붙여넣기만** 사용합니다.

트랙을 고르고 설계 메모를 씁니다

	설계	구현	검증
기본 트랙	본인	**AI**	본인 (반례, 속도)
도전 트랙	본인	본인	본인 + AI 리뷰

```
문제:          (한 줄로)
키(기준):      (무엇이 같으면 같은 것으로 볼까?)
자료구조:      (HashMap? 정렬? 왜?)
경계 규칙:     (시작일·종료일 포함? 상실일이 비면?)      ← 이번에 추가
예상 복잡도: (O(?))
```

함수 완성의 기준

- [ ] `small/` 데이터 테스트를 통과한다
- [ ] 반례를 **1개 이상** 직접 만든다 — 이번에는 **경계값 체크리스트에서 최소 1개**
- [ ] `large/` 데이터에서 **수 초 안에** 끝난다 (이 장의 풀이는 모두 약 15ms 이하)

먼저 끝났다면 도전 과제

1. **겹침이 A-B-C로 연쇄될 때 한 묶음으로 출력**
   6.2 구간 병합 응용

2. **가입자별 자격 공백 일수의 합계**
   병합한 자격 기간 사이의 빈 날

3. **스위핑으로 동시 입원 환자 수가 가장 많은 날**
   +1 / −1 사건

4. **분기·반기별 집계를 투 포인터로 구현하고 시간 비교**
   기간마다 전체를 훑는 방식과 비교

Day 2 정리

오늘 점검기에
네 기능이 더 붙었습니다

특정 기간 조회 · 자격 기간 밖 진료 · 중복 입원 · 월별 집계

내일은 **상위 N건** · **동일인 판별** · **튀는 청구**를 찾고, 3일 동안 만든 기능을 하나로 합칩니다.

국민건강보험공단 전산전문교육 · 미니 프로젝트 기반 알고리즘 실전 학습

Day 3
추출·탐지와 완성

힙 · 문자열 유사도 · 이상치 탐색 · 점검기 통합

9.

힙

전부 정렬하지 않고
필요한 것만 뽑습니다

100만 건에서 10건만 필요합니다

전체 정렬 후 10개

약 580ms

청구 100만 건

힙으로 10개

약 8ms

청구 100만 건

Apple M1, JDK 21 측정값(정렬 570~590ms). 교육장 노트북에서는 값이 다릅니다.

정렬은 필요 이상의 일을 합니다

필요한 건 10개인데, 나머지 99,990건의 순서까지 전부 맞추는 셈입니다. 정렬은 O(n log n)입니다.

데이터	전체 정렬 후 10개	힙으로 10개
청구 100,000건	약 47ms	약 1ms
청구 1,000,000건	약 570~590ms	약 8ms

> 건수가 늘수록 차이가 커집니다. 힙은 **O(n log N)** (N = 뽑을 개수, 10이면 log N은 약 3)

힙은 "가장 작은 것이 맨 위"인 구조

최소 힙

- 부모가 자식보다 작거나 같음
- 그래서 **가장 작은 값이 항상 맨 위**
- 맨 위 확인 **O(1)**
- 넣기·빼기 **O(log n)**

우선순위 큐

- "우선순위가 높은 것부터 꺼내는 대기열"
- 힙으로 만든 것이 가장 흔함
- 급한 업무부터 처리하는 대기열도 같은 구조

구조 내부(배열로 트리를 표현하는 방식, 올리고 내리는 과정)는 다루지 않습니다. 사용법과 비용을 이해하는 데 집중합니다.

상위 N개를 지키는 방법 (N = 3)

최소 힙에 N개만 담고, 새 값이 맨 위(지금 상위 N개 중 가장 작은 값)보다 크면 맨 위를 내보내고 새 값을 넣습니다. 진료비 `[50, 90, 20, 70, 80, 10]`

읽은 값	힙 (맨 위가 가장 작음)	설명
50	{50}	3개 미만 → 넣음
90	{50, 90}	넣음
20	{20, 50, 90}	넣음 (3개 참)
70	{50, 70, 90}	70 > 맨 위 20 → 20을 빼고 70 넣음
80	{70, 80, 90}	80 > 맨 위 50 → 50을 빼고 80 넣음
10	{70, 80, 90}	10 < 맨 위 70 → 버림

결과: 90, 80, 70. 힙은 늘 N개만 담으므로 한 건마다 O(log N)입니다. N이 n에 가까우면(예: 절반) 이득이 없으니 그냥 정렬하세요.

Java의 PriorityQueue

```
PriorityQueue<Long> heap = new PriorityQueue<>();    // 기본이 최소 힙
heap.offer(50);                                      // 넣기  O(log n)
heap.peek();                                         // 맨 위 확인  O(1)
heap.poll();                                         // 맨 위 빼기  O(log n)
```

객체를 담을 때는 Comparator로 "무엇이 더 작은가"를 정해 줍니다.

주의: PriorityQueue를 그냥 출력하거나 반복하면 **정렬된 순서가 아닙니다.** 순서대로 꺼내려면 poll()을 반복하거나 꺼낸 목록을 정렬해야 합니다.

🔧 진료비 상위 N건 추출

규칙: 진료비가 큰 순서. **진료비가 같으면 청구ID 오름차순.** N이 0 이하면 빈 목록, 전체 건수보다 크면 전체.

```
static final Comparator<Outpatient> BY_FEE_DESC =
        Comparator.comparingLong(Outpatient::fee).reversed().thenComparing(Outpatient::claimId);

public static List<Outpatient> topN(List<Outpatient> claims, int n) {
    if (n <= 0) return List.of();
    // 힙의 맨 위에 "상위 N개 중 가장 약한 청구"가 오도록 순서를 뒤집는다
    PriorityQueue<Outpatient> heap = new PriorityQueue<>(BY_FEE_DESC.reversed());
    for (Outpatient c : claims) {
        if (heap.size() < n) heap.offer(c);
        else if (BY_FEE_DESC.compare(c, heap.peek()) < 0) {   // c가 맨 위보다 더 "강하면"
            heap.poll();
            heap.offer(c);
        }
    }
    List<Outpatient> result = new ArrayList<>(heap);
    result.sort(BY_FEE_DESC);                                 // 힙은 순서대로 나오지 않으므로 정렬
    return result;
}
```

시간 복잡도 O(n log N). 부호가 헷갈리기 쉬운 부분(`compare(...) < 0`)이라 반드시 반례로 확인합니다.

이렇게 검증했습니다

검증 (전체 정렬 후 앞에서 N개 자른 결과와 비교)	결과
N = 1, 10, 100	완전히 일치
N = 100,000 (전체와 같음), 200,000 (전체보다 큼)	일치
N = 0, −3	빈 목록
진료비가 같은 청구가 여러 건 (동점)	일치 (청구ID 오름차순)

Java 21에서 `--release 17`로 컴파일해서 실행했습니다.

상위 10건을 보니 이상합니다

순위	청구ID	진료일	진료비(원)
1	C0095477	2026-02-21	2,925,400
2	C0095462	2026-09-27	2,835,600
3	C0095474	2026-03-07	2,817,000
…			
10			2,587,900

보통 청구는 3,000원 ~ 30만 원대인데, 상위 10건은 모두 **250만 원이 넘습니다.** "이 중 이상한 청구는 몇 건일까?" → 11장으로 이어집니다.

10.

문자열 유사도

표기가 조금 다른 이름,
같은 사람일까요

김가온과 김가운은 같은 사람일까요?

자격 이력에 표기가 조금씩 다른 가입자가 있습니다

equals는 "다르다"고만 말합니다

표기 A	표기 B	차이	equals
김가온	김가운	글자 하나가 다름	false
김가온	김 가온	공백이 들어감	false
김가온	김가	글자 하나가 빠짐	false

세 경우 모두 "다르다"는 답뿐입니다. **얼마나 다른가**를 숫자로 말해 주는 방법이 필요합니다.

편집 거리: 바꾸는 데 필요한 최소 횟수

편집	예
삽입	김가 → 김가온
삭제	김가온 → 김가
치환	김가온 → 김가운

> 김가온 → 김가운 = **1** (치환 한 번). 거리가 작을수록 비슷하고, 0이면 완전히 같습니다. (Levenshtein 거리)

DP: 작은 문제의 답을 표에 적어 둡니다

1. **부분 문제를 정의한다**
   `dp[i][j]` = A의 앞 i글자를 B의 앞 j글자로 바꾸는 최소 횟수

2. **점화식을 세운다**
   마지막 글자만 보고, 이미 구한 더 작은 문제로 환원

3. **시작값과 채우는 순서를 정한다**
   `dp[i][0] = i`, `dp[0][j] = j`

```
dp[i][j] = min( dp[i-1][j]   + 1,                    // A의 i번째 글자를 삭제
                dp[i][j-1]   + 1,                    // B의 j번째 글자를 삽입
                dp[i-1][j-1] + (같은 글자면 0, 다르면 1) )  // 치환
```

표를 채워 봅시다: 김가온 → 김가운

	(빈)	김	가	운
(빈)	0	1	2	3
김	1	0	1	2
가	2	1	0	1
온	3	2	1	**1**

오른쪽 아래 칸이 답입니다: **1**. 마지막 칸(온 vs 운)은 글자가 달라서 `dp[2][2] + 1 = 1`이 가장 작습니다. 이 표는 **같은 계산을 다시 하지 않기 위한 장치**입니다.

편집 거리 코드

```
static int editDistance(String a, String b) {
    int[][] dp = new int[a.length() + 1][b.length() + 1];
    for (int i = 0; i <= a.length(); i++) dp[i][0] = i;
    for (int j = 0; j <= b.length(); j++) dp[0][j] = j;
    for (int i = 1; i <= a.length(); i++) {
        for (int j = 1; j <= b.length(); j++) {
            int cost = a.charAt(i - 1) == b.charAt(j - 1) ? 0 : 1;
            dp[i][j] = Math.min(Math.min(dp[i - 1][j] + 1, dp[i][j - 1] + 1), dp[i - 1][j - 1] + cost);
        }
    }
    return dp[a.length()][b.length()];
}
```

시간 복잡도 O(A의 길이 × B의 길이). 이름은 몇 글자뿐이라 한 번은 순식간입니다. 글자 단위는 한글 음절 하나(char)이며, 자모 단위로 쪼개 비교하지는 않습니다.

모든 쌍을 비교하면 2억 쌍

모든 쌍 비교

199,990,000

가입자 20,000명의 쌍

같은 생년월일끼리만

8,951

해시로 묶은 뒤 묶음 안에서만

동일인이라면 **생년월일은 같을 것**이라고 가정하고, 3장의 해시로 생년월일별로 묶습니다. 비교 횟수가 약 **22,000분의 1**로 줄었습니다.

🔧 동일인·중복 데이터 판별 코드

```
public static List<Suspect> findSimilarMembers(List<Qualification> quals, int maxDistance) {
    Map<String, Qualification> members = new LinkedHashMap<>();
    for (Qualification q : quals) members.putIfAbsent(q.memberId(), q);       // 가입자별 한 번만

    Map<LocalDate, List<Qualification>> byBirth = new HashMap<>();            // 생년월일로 묶기 (해시)
    for (Qualification q : members.values())
        byBirth.computeIfAbsent(q.birthDate(), k -> new ArrayList<>()).add(q);

    List<Suspect> result = new ArrayList<>();
    for (List<Qualification> g : byBirth.values())
        for (int i = 0; i < g.size(); i++)
            for (int j = i + 1; j < g.size(); j++) {                          // 같은 묶음 안에서만
                int d = editDistance(normalize(g.get(i).name()), normalize(g.get(j).name()));
                if (d <= maxDistance) result.add(new Suspect(g.get(i).memberId(), g.get(j).memberId(), d));
            }
    return result;
}
static String normalize(String name) { return name.replace(" ", ""); }       // 공백 제거
```

규칙: 생년월일이 같고, 서로 다른 memberId이며, 공백을 뺀 이름의 편집 거리가 **1 이하**이면 동일인 **의심** 쌍. 20,000명에서 약 7ms, **21쌍**.

결과와 검증

가입자 A	가입자 B	편집 거리	해석
M000118	M019997	1	글자 하나가 다르거나 빠짐
M000107	M019986	1	글자 하나가 다르거나 빠짐
M000108	M019987	0	공백만 다름 (공백 제거 후 같음)

검증	결과
편집 거리: 김가온/김가운 = 1, kitten/sitting = 3	일치
빈 문자열 (빈↔abc = 3, 빈↔빈 = 0)	일치
생년월일로 묶은 결과 vs 모든 쌍 비교 (일부 데이터)	건수 일치
심어 둔 동일인 21쌍	정확히 찾음

의심은 의심일 뿐입니다

동명이인

생년월일과 이름이 같은 **다른 사람**이 있을 수 있습니다. 확정은 사람이 합니다.

임계값 선택

1 이하는 오탐이 적지만 많이 다른 표기는 놓칩니다. 2 이하로 넓히면 의심 쌍이 늘어납니다. 어디까지를 의심으로 볼지는 **업무 기준**이 정합니다.

> 생년월일이 틀린 경우는 이 방식으로 찾지 못합니다 (묶음이 달라지므로).

11.

이상치 탐색

튀는 값을 감이 아니라
기준선으로 찾습니다

얼마부터가 "튄다"고 할 수 있을까요?

100만 원? 50만 원? 감으로 정하면 맞는 기준인지 알 수 없습니다

평균은 튀는 값에 끌려갑니다

값	평균	중앙값
1, 2, 3, 4, 5	3	3
1, 2, 3, 4, 500	**102**	3

튀는 값 하나가 평균을 크게 움직입니다. 기준선을 평균으로 잡으면 **기준선도 같이 튑니다.** 중앙값(정렬했을 때 가운데 값)은 거의 영향을 받지 않습니다.

정렬해서 위치로 구합니다

이름	위치	뜻
Q1	25% 지점	아래에서 4분의 1
중앙값 (Q2)	50% 지점	가운데
Q3	75% 지점	위에서 4분의 1
IQR	Q3 − Q1	가운데 절반이 퍼져 있는 폭

기준선 (IQR 방법): `Q3 + 1.5 × IQR` 보다 크거나 `Q1 − 1.5 × IQR` 보다 작으면 이상치. 1.5는 통계에서 널리 쓰는 관례이며 업무에 맞게 조정합니다.

우리 데이터의 기준선

항목 (청구 100,000건의 진료비)	값(원)
Q1	78,700
중앙값	153,100
Q3	228,500
IQR	149,800
상한 (Q3 + 1.5 × IQR)	**453,200**
하한 (Q1 − 1.5 × IQR)	−146,000 (진료비는 음수가 없어 의미 없음)

진료비가 **453,200원**보다 크면 튀는 청구입니다.

분위수 코드

```
static double quantile(long[] sorted, double p) {
    double idx = (sorted.length - 1) * p;                  // 예: 5개의 중앙값 → idx = 2.0
    int lo = (int) Math.floor(idx), hi = (int) Math.ceil(idx);
    return sorted[lo] + (sorted[hi] - sorted[lo]) * (idx - lo);   // 이웃한 두 값 사이를 선형 보간
}
```

위치가 정수가 아닐 때 이웃한 두 값 사이를 **선형 보간**합니다. 분위수의 정의는 여러 가지이므로 이 과정에서는 이 방법으로 통일합니다.

확인한 값: `[1,2,3,4,5]`의 중앙값 = 3, `[1,2,3,4]`의 중앙값 = 2.5

하루 청구 건수가 갑자기 늘어난 날

"평소"를 **직전 7일의 평균**으로 보고, 당일 건수가 평소의 3배를 넘으면 급증일입니다.

매일 직전 7일을 다시 더하면

하루마다 덧셈 7번. 창이 30일, 90일로 길어질수록 느려집니다.

O(n × 창 크기)

창을 한 칸씩 밀면

들어오는 값을 더하고 나가는 값을 뺍니다. 창 크기와 무관합니다.

O(n)

창을 한 칸씩 밀면서

```
창(3일) = [3, 1, 4]   합 8
다음 날 1이 들어오고 3이 나감 → 합 = 8 + 1 − 3 = 6      (창 = [1, 4, 1])
```

한 칸마다 덧셈·뺄셈 한 번이라 **O(n)**입니다. 7장의 누적합과 비슷하지만, 창이 **움직이며 실시간으로 갱신**된다는 점이 다릅니다.

슬라이딩 윈도우 코드

```
static List<LocalDate> spikeDays(long[] daily, LocalDate base, int window, double factor) {
    List<LocalDate> result = new ArrayList<>();
    if (daily.length <= window) return result;
    long windowSum = 0;
    for (int i = 0; i < window; i++) windowSum += daily[i];          // 첫 창
    for (int i = window; i < daily.length; i++) {
        double avg = (double) windowSum / window;                    // 직전 window일의 평균
        if (avg > 0 && daily[i] > factor * avg) result.add(base.plusDays(i));
        windowSum += daily[i] - daily[i - window];                   // 창을 한 칸 밀기
    }
    return result;
}
```

처음 window일은 비교할 "직전"이 없어 판단하지 않고, 평균이 0이면 건너뜁니다. **한계:** 급증일 직후에는 평균이 높아져서 급증이 이틀 연속이면 둘째 날을 놓칠 수 있습니다.

🔧 튀는 청구 찾기 — 두 기준

기준	방법	찾는 것
진료비가 기준선 밖	분위수 (11.1)	비정상적으로 큰 **청구**
날짜별 건수가 평소의 3배 초과	슬라이딩 윈도우 (11.2)	청구가 몰린 **날**

```
public static List<String> feeOutliers(List<Outpatient> claims) {
    long[] fees = claims.stream().mapToLong(Outpatient::fee).sorted().toArray();
    double q1 = quantile(fees, 0.25), q3 = quantile(fees, 0.75), iqr = q3 - q1;
    double low = q1 - 1.5 * iqr, high = q3 + 1.5 * iqr;
    List<String> result = new ArrayList<>();
    for (Outpatient c : claims) if (c.fee() < low || c.fee() > high) result.add(c.claimId());
    return result;
}
// 급증일:  spikeDays(dailyCounts(claims, BASE, 273), BASE, 7, 3.0)
```

이상 청구 **44건**(약 13ms), 급증일 **3일**(약 0.01ms)을 찾았습니다.

급증일 결과

급증일	당일 건수	직전 7일 평균	기준 (평균 × 3)
2026-02-20	1,876건	350.7건	1,052.1건
2026-05-11	1,870건	337.9건	1,013.6건
2026-07-30	1,894건	343.7건	1,031.1건

> 급증일을 제외한 날의 하루 최대 청구 건수는 **401건**입니다. 기준(약 1,000건)에 한참 못 미쳐서 급증일이 아닌 날은 잡히지 않았습니다.

검증	결과
분위수 중앙값 [1..5] = 3, [1..4] = 2.5	일치
급증일: 입력이 창보다 짧음 / 평균이 0	빈 목록 / 건너뜀
심어 둔 이상 청구 44건, 급증일 3일	정확히 찾음

이상치는 오류가 아닙니다

사람이 확인할 후보

기준선 밖이라고 잘못된 청구는 아닙니다. 큰 수술이나 장기 입원은 정상적으로 큰 진료비입니다.

기준선의 한계

전체 하나의 기준선은 진료 종류가 섞이면 부정확합니다. 요양기관 종별이나 진료 유형별로 따로 만들면 더 정확합니다.

이 데이터는 진료비를 균등하게 만들어서 기준선 밖의 값이 심어 둔 것뿐입니다. **실제 진료비는 오른쪽으로 길게 늘어진 분포라서 정상 청구도 상한을 넘는 경우가 많습니다.** 1.5배, 7일, 3배는 관례이며 업무 감각에 맞게 조정합니다.

12.

Sprint 3

3일 동안 만든 기능을
하나의 점검기로 완성합니다

이제 하나로 합칩니다

3일 동안 만든 함수 9개

지금까지 만든 함수 9개

함수	입력	출력	장
`duplicateClaims`	진료 청구	청구ID 목록	3.3
`noQualificationClaims`	진료 청구, 자격 이력	청구ID 목록	3.4
`findByPeriod`	정렬된 청구, 시작일, 종료일	청구 목록	5.3
`outOfQualificationPeriod`	진료 청구, 자격 이력	청구ID 목록	6.3
`overlappingAdmissions`	입원 청구	청구ID 쌍 목록	6.4
`monthlySummary`	진료 청구	월별 건수·합계	7.3
`topN`	진료 청구, N	청구 목록	9.2
`findSimilarMembers`	자격 이력, 허용 거리	가입자 쌍 목록	10.2
`feeOutliers` , `spikeDays`	진료 청구	청구ID 목록, 날짜 목록	11.3

Main에서 합칩니다

```
public class Main {
    public static void main(String[] args) {
        List<Qualification> quals = DataLoader.loadQualifications("data/large/qualifications.csv");
        List<Outpatient> outs   = DataLoader.loadOutpatients("data/large/outpatient.csv");
        List<Inpatient> ins     = DataLoader.loadInpatients("data/large/inpatient.csv");

        List<String> dup          = DuplicateClaims.find(outs);
        List<String> noQual       = NoQualificationClaims.find(outs, quals);
        List<String> outPeriod    = OutOfQualificationPeriod.find(outs, quals);
        List<OverlapPair> overlap = OverlappingAdmissions.find(ins);
        List<Outpatient> top10    = TopN.find(outs, 10);
        List<Suspect> similar     = SimilarMembers.find(quals, 1);
        List<String> outliers     = FeeOutliers.find(outs);
        List<LocalDate> spikes    = SpikeDays.find(outs, 7, 3.0);

        ReportPrinter.print(outs.size(), quals, dup, noQual, outPeriod, overlap, top10, similar, outliers, spikes);
    }
}
```

합칠 때 주의할 점

주의	이유
정렬은 한 번만	정렬을 전제로 하는 함수에 정렬된 목록을 넘기고, 각 함수가 따로 정렬하느라 시간을 낭비하지 않게
원본을 바꾸지 않기	한 함수가 목록을 정렬·수정하면 뒤의 함수 결과가 달라질 수 있음. 정렬할 때는 복사본
한 청구가 여러 항목에 나올 수 있음	"중복 청구"이면서 "이상 청구"일 수 있어 건수를 단순히 합치면 안 됨
기준이 함수마다 다르지 않게	시작일·종료일 포함, 상실일이 비면 유지 중 같은 규칙은 모든 함수에서 같아야 함

실행 결과

```
[점검 결과] 청구 100,000건 / 가입자 20,000명
- 중복 청구: 37건
- 자격 없는 사람의 청구: 12건
- 자격 기간 밖 진료: 58건
- 중복 입원 기간: 9건
- 진료비 상위 10건: 2,925,400원 ~ 2,587,900원
- 동일인 의심 쌍: 21쌍
- 이상 청구: 44건
- 청구 급증일: 3일 (2026-02-20, 2026-05-11, 2026-07-30)
```

점검 로직 전체 실행은 약 70~90ms였습니다 (파일 읽기 제외, Apple M1). 1.3에서 이중 반복문 하나가 5분 넘게 걸렸던 것과 비교해 보세요.

정답을 공개합니다

가상 데이터를 만들 때 **심어 둔 오류**의 개수입니다. 지금까지 공개하지 않았습니다.

점검	심어 둔 개수	찾은 개수
중복 청구	37	37
자격 없는 사람의 청구	12	12
자격 기간 밖 진료	58	58
중복 입원 기간	9	9
동일인 의심 쌍	21	21
이상 청구	44	44
청구 급증일	3	3

개수가 다르게 나왔다면 **그 차이가 반례를 찾는 단서**입니다. 어느 기준(규칙)에서 달라졌는지 따져 봅니다.

이번에는 사내 LLM이라고 가정합니다

Key Scene: **"사내 LLM에서도 이렇게 사용하면 되겠다!"**

항목	지금까지	이번 Sprint
AI에게 주는 것	코드, 데이터 일부	**규칙과 구조 중심.** 개인정보 대신 가공한 예시
AI가 하는 것	코드 작성·리뷰	**질문에 답하기, 검색** (코드를 실행하지 못함)
실행과 확인	AI가 대신 확인할 때도 있음	**전부 내가** (IDE에서 직접 실행)
코드 가져오기	복사·붙여넣기	복사·붙여넣기 (파일을 직접 읽게 하지 않음)

실제 값 대신 가공한 값을 씁니다

원본	질문에 쓸 때
M000118 / 김가온 / 1985-03-12	M1 / 가입자A / 날짜는 상대값 (D+3)
실제 진료비	비슷한 크기의 임의 숫자

> 이 Sprint에서는 가상 데이터이지만 **같은 습관으로** 연습합니다. 어디까지 붙여도 되는지는 사내 보안 정책이 정합니다.

이번 Sprint에서 만들 것

함수	설명	장
`topN`	진료비 상위 N건	9.2
`findSimilarMembers`	동일인·중복 의심 쌍	10.2
`feeOutliers` , `spikeDays`	튀는 청구	11.3
통합	Day 1~2 기능과 함께 `Main` 에 합치기	12.1

> 설계 메모는 Day 2와 같은 양식입니다. (문제 / 키 / 자료구조 / 경계 규칙 / 예상 복잡도)

함수 완성의 기준

- [ ] `small/` 데이터 테스트를 통과한다
- [ ] 반례를 **1개 이상** 직접 만든다
- [ ] `large/` 데이터에서 **수 초 안에** 끝난다
- [ ] 통합 점검기가 모든 항목을 출력한다
- [ ] 이번 Sprint에서 AI에게 한 질문 중 **사내 LLM에서도 그대로 쓸 수 있는 것 3개**를 골라 둔다

먼저 끝났다면 도전 과제

1. **요양기관별(또는 종별) 기준선을 따로 계산해서 이상 청구 찾기**
   11장 확장
2. **동일인 판별의 임계값을 1과 2로 바꿔서 의심 쌍이 몇 개 늘어나는지 비교**
3. **점검 결과를 가입자별로 묶어서 한 가입자에게 몇 개 항목이 걸렸는지 보기**
   `HashMap`
4. **한 청구가 여러 점검에 걸린 경우를 찾아 "중복 등장 청구" 목록 만들기**

3일 동안 만든 것

날	만든 것	배운 것
Day 1	중복 청구, 자격 없는 사람의 청구	시간 복잡도, 해시, 힌트 질문법
Day 2	특정 기간 조회, 자격 기간 밖 진료, 중복 입원, 월별 집계	정렬·이분 탐색, 구간, 누적합·투 포인터, 리뷰·반례 질문법
Day 3	상위 N건, 동일인 판별, 튀는 청구, **통합 점검기**	힙, 편집 거리·DP, 분위수·슬라이딩 윈도우, 사내 LLM 조건

이 과정에서 가져갈 것

1. **문제를 쪼개는 힘**
   이건 대조 문제, 이건 기간 문제, 이건 추출 문제
2. **효율을 판단하는 힘**
   코드를 보고 증가 속도를 읽는다
3. **검증하는 힘**
   규칙을 정하고 반례로 확인한다
4. **AI를 다루는 방법**
   규칙을 먼저 주고, 힌트와 리뷰를 받고, 실행과 판단은 내가 한다

사내에 돌아가서 이렇게

1. 부록 A의 질문 템플릿을 NHIS-MATE에서 그대로 써 봅니다
2. 부록 B의 선택 치트시트를 옆에 두고 "찾기 / 대조하기 / 집계하기 / 골라내기" 중 무엇인지부터 정합니다
3. 부록 C의 복습 로드맵과 부록 D의 연습 문제로 이어서 연습합니다

3일 정리

청구 데이터 점검기가 완성되었습니다

찾고 · 대조하고 · 집계하고 · 골라내는 알고리즘을
직접 만들고, 검증하고, AI와 함께 다뤘습니다.

A–D

부록

사내에 돌아가서
계속 쓸 수 있는 자료

A. 질문의 기본 틀

```
[규칙]  기간은 시작일과 종료일을 모두 포함한다.   (필요한 규칙을 줄 단위로)
[입력]  List<Outpatient>: claimId, memberId, institution, visitDate, fee
[출력]  중복 청구의 claimId 목록
[요청]  (다음 쪽의 상황별 요청)
[제한]  코드는 주지 마. / 3가지만. / 표로.
```

① 규칙을 먼저 줍니다

기준 없이 시키면 AI가 추측해서 답합니다 (1.3의 교훈)

② 원하는 형태를 말합니다

"코드는 주지 마", "표로", "3가지만"

③ 실행과 판단은 내가 합니다

AI의 답은 후보이며 규칙에 비추어 내가 확인합니다

④ 실제 개인정보는 넣지 않습니다

가공한 예시로 바꿉니다

A. 상황별 템플릿 ①

① 문제 쪼개고 알고리즘 고르기

```
[규칙/입력/출력]
찾기/대조하기/집계하기/골라내기 중
어디에 가까운지 알려 주고, 쓸 만한
알고리즘 후보를 2개만 비교해 줘.
코드는 주지 마.
```

② 힌트 받기 (정답 대신)

```
힌트를 3단계로 줘.
방향 → 자료구조 → 의사코드.
내가 "다음"이라고 할 때만 다음 단계를
줘. 코드는 쓰지 마.
```

③ 내 설계 확인받기

```
내 설계는 이거야: [문제 / 키 / 자료구조
/ 경계 규칙 / 예상 복잡도]
맞는 방향인지만 말해 줘. 틀렸다면
어디가 문제인지만 짚고, 고친 코드는
주지 마.
```

④ 코드 리뷰 (규칙 기준)

```
[규칙]
아래 코드가 이 규칙대로 동작하는지
검토해 줘. 규칙과 다르게 동작하는
부분만 줄 번호와 함께 알려 줘.
```

A. 상황별 템플릿 ②

⑤ 반례 만들기

```
이 함수가 틀릴 만한 입력을 7개
만들어 줘. 경계값 위주로.
각 입력마다 "내가 기대하는 결과"와
"이 입력을 고른 이유"를 적어 줘.
```

⑥ 시간 복잡도 점검

```
이 코드의 시간 복잡도를 설명해 줘.
숨은 반복문이 있다면 짚어 줘.
데이터가 10만 건, 100만 건일 때
문제가 없을지도 알려 줘.
```

⑦ 테스트 케이스 만들기

```
이 규칙으로 JUnit 테스트 케이스를
만들어 줘. 정상 1개, 경계 3개,
예외 1개.
```

⑧ 코드 설명 받기

```
이 코드가 하는 일을 한 줄씩 설명해
줘. 입력이 이렇게 들어오면 단계마다
값이 어떻게 변하는지도 보여 줘.
[입력 예시 (가공한 값)]
```

A. 상황별 템플릿 ③ · 질문에 넣지 말 것

⑨ 에러 해석

```
이 에러가 났어: [에러 메시지]
원인 후보를 3가지만 알려 줘. 수정 코드는 주지 마.
```

넣지 않는다	대신
실제 가입자ID, 이름, 주민등록번호, 생년월일	M1, 가입자A, 임의 날짜
실제 진료일, 청구 금액	상대 날짜(D+3), 비슷한 크기의 임의 숫자
내부 서버 주소, 계정, 접속 정보	생략 또는 [서버]
업무 규정 원문 전체	필요한 규칙만 요약

어디까지 붙여도 되는지는 **사내 보안 정책이 정합니다.** 이 표는 안전한 쪽으로 정리한 예시이며 사내 정책이 우선입니다.

B. 이런 문제엔 이 알고리즘

이런 문제	알고리즘	자료구조	시간	장
"있는지 없는지", 두 데이터 대조	해시	`HashSet`	O(m + n)	3
같은 것 찾기, 중복 / 키별로 세기	해시	`HashSet` , `HashMap`	O(n)	3, 7
정렬된 데이터에서 기간·값 찾기	이분 탐색	정렬된 `List`	O(log n)	5
두 기간이 겹치는지	구간 겹침 판정	-	O(1)	6
겹치는 기간 합치기	구간 병합	정렬	O(n log n)	6
같은 가입자의 겹치는 기간	정렬 + 훑기	`Map` + 정렬	O(n log n)	6
구간 합을 여러 번	누적합	배열	조회 O(1)	7
정렬된 두 목록을 함께 훑기	투 포인터	정렬된 `List`	O(n + m)	7
상위 N개만 필요	힙	`PriorityQueue`	O(n log N)	9
표기가 다른 문자열이 비슷한가	편집 거리 (DP)	2차원 배열	O(길이²)	10
기준선 밖 값 찾기	분위수 (IQR)	정렬	O(n log n)	11
움직이는 구간의 합·평균	슬라이딩 윈도우	배열	O(n)	11

B. 코드를 보기 전에 묻는 질문

1. **지금 하려는 일은 찾기 / 대조하기 / 집계하기 / 골라내기 중 무엇인가?**
2. **데이터는 얼마나 커지는가? (1만 건? 100만 건?)**
3. **같은 일을 몇 번 하는가? (한 번? 여러 번?)**
   정렬·누적합이 이득인지 결정
4. **정렬되어 있는가? 정렬해도 되는가?**
5. **"같다"의 기준과 "기간"의 경계 규칙은 정해져 있는가?**

반복문이 두 겹으로 보일 때: 안쪽이 **목록 찾기**면 해시로, **범위 찾기**면 정렬 + 이분 탐색으로, **구간 합**이면 누적합으로 바꿀 수 있는지 확인합니다.

B. 경계값 체크리스트

- [ ] 시작일 = 종료일 (하루짜리 기간)
- [ ] 퇴원일 = 다음 입원일 (하루 겹침)
- [ ] 하루 차이로 안 겹치는 경우
- [ ] 한 기간이 다른 기간을 통째로 포함
- [ ] 상실일이 비어 있는 경우
- [ ] 빈 입력, 데이터 1건 · 같은 날짜·같은 값이 여러 건
- [ ] 연말·연초, 윤년(2/29)

C. 복습 로드맵

단계	할 일	확인할 것
1. 오늘 만든 것 다시 만들기	점검기를 AI 없이 처음부터 다시 구현해 보기 (도전 트랙 방식)	설계 메모만 보고 코드를 쓸 수 있는가
2. 같은 유형 문제 풀기	부록 D의 장별 문제를 풀고, 풀기 전에 **어떤 알고리즘인지 먼저 적기**	문제를 보고 알고리즘을 고르는 속도
3. 내 업무에 적용하기	업무 중 느린 처리나 중복·겹침 확인을 하나 골라 개선	개선 전후의 시간을 직접 재 보기
4. 다음 주제로 넓히기	스택·큐, 그래프(BFS/DFS), 그리디, DP 확장	점검기 기능과 연결되는 사례 찾기

기간(몇 주) 대신 단계로 정리했습니다. 각자의 속도에 맞춰 진행합니다.

C. 추천 도서

『누구나 자료 구조와 알고리즘』

- 출판사: 길벗
- 원서: Jay Wengrow, *A Common-Sense Guide to Data Structures and Algorithms*의 번역서로 확인됨
- 수학 증명보다 **직관**으로 설명하는 입문서

이 과정과 함께 보면

- 시간 복잡도 → 해시 → 정렬·탐색 → 힙이 비슷한 흐름
- 모르는 개념이 나올 때 **찾아보는 용도**
- 개정판이 나온 것으로 보이니 구입 시 판본을 확인

도서 링크: https://product.kyobobook.co.kr/detail/S000001834743

D. 프로그래머스 연습 문제 (장별)

장	주제	후보 문제
3	해시	완주하지 못한 선수 / 전화번호 목록 / 폰켓몬 / 의상
5	정렬 · 이분 탐색	K번째수 / 가장 큰 수 / H-Index / 입국심사
6	구간	단속카메라 / 과제 진행하기 / 광고 삽입
7	투 포인터 · 누적합	연속된 부분 수열의 합 / 구명보트 / 보석 쇼핑
9	힙	더 맵게 / 디스크 컨트롤러 / 이중우선순위큐
10	문자열 유사도 · DP	뉴스 클러스터링 / 정수 삼각형
11	슬라이딩 윈도우	보석 쇼핑 / 연속된 부분 수열의 합

풀기 전에 **부록 B의 질문**으로 알고리즘을 먼저 정하고, 푼 뒤에는 시간 복잡도를 적어 봅니다. AI는 막혔을 때 **힌트 질문(부록 A ②)**으로만 씁니다.